

PopOut: Monte Carlo Tree Search and ID3 Decision Trees

Artificial Intelligence 2025/2026

Miguel Almeida Francisco Passos Pedro Resende

Faculty of Sciences of the University of Porto

17 May 2026

The problem

PopOut: a Connect-4 variant

Moves:

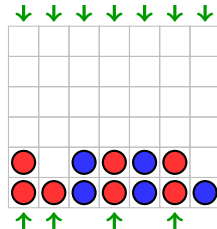
- *drop* (place a piece on top) or *pop* (remove your own piece from the bottom row)

Three special rules:

- Simultaneous four-in-a-rows → the popper wins
- Full board → player to move chooses pop or draw
- 3-fold repetition → either player can claim a draw

Goals of the work:

- Implement two agents: **MCTS** (adversarial search) and **ID3** (decision tree)
- Support 3 scenarios: human-vs-human, human-vs-computer, computer-vs-computer (different algorithms)



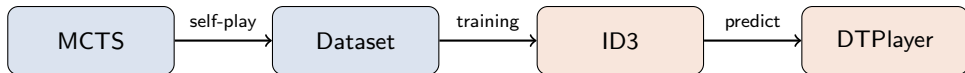
Red to move.

Top arrows: available *drops* (all columns have space).

Bottom arrows: available *pops* (columns with a red piece at the bottom).

Our approach

Two-phase pipeline



MCTS phase (adversarial search)

- From-scratch algorithm with UCT
- Numba JIT on the rollout ($\sim 50\times$ speedup)
- Empirical tuning via tournaments

ID3 phase (supervised learning)

- Dataset generated by self-play of the optimised MCTS
- Tree trained with empirically-validated pipeline
- DTPlayer integrates the trained tree into the game engine

MCTS — Implementation

Four phases per iteration:

- **Selection** — descend the tree via UCT
- **Expansion** — add one (or more) children for an untried move
- **Simulation** — random rollout to a terminal state
- **Backpropagation** — update visit count and reward

UCT formula

$$\text{UCT}(c) = \bar{X}_c + c_{\text{explore}} \sqrt{\frac{\ln N_p}{N_c}}$$

At opponent-to-move nodes the exploitation term is mirrored ($1 - \bar{X}_c$) — the opponent picks the move that hurts us.

Additional variants implemented (beyond standard MCTS):

- `num_children_to_expand` (k): expand up to k children per visit
- `uct_variant`: classic UCB1 or UCB1-Tuned (variance-aware)

MCTS — Hyperparameter tuning

Four winrate sweeps with Wilson 95% CI¹

Methodology:

- 100–400 games per matchup, P1/P2 alternated 50/50
- base_seed=0 → bit-identical reproducible

Sweeps:

- **Iterations** $\{200, 1k, 5k, 20k, 100k\}$
→ **20k** (diminishing returns after this point)
- **Exploration c** $\{0.1, 0.25, 0.5, \sqrt{2}, 2, 3, 4, 5\}$
→ $c = 2$ (optimum > textbook $\sqrt{2}$)
- **Fan-out k** $\{1, 3, 7, 10, 14\}$ + direct $k = 1$ vs $k = 7$
→ $k = 1$ (wins 68–32 directly)
- **UCT variant:** UCB1 vs UCB1-Tuned
→ **UCB1** at 20k iter (wins 237–163 in N=400)

Final operating point

Iterations	20 000
Exploration c	2.0
Fan-out k	1
Variant	UCB1

This configuration wins **66–34** against the baseline (5k iter, $c = \sqrt{2}$).

Next slide: counterintuitive result in the UCB1 vs UCB1-Tuned comparison.

¹Wilson 95% CI: confidence interval for binomial proportions. Statistical standard for reporting winrates. 

Interesting finding — non-transitivity

Non-transitivity: “A beats B more than C beats B” does **not** imply “A beats C”. Analogous to rock-paper-scissors, but in game strategies.

Matchup	Winrate (A)	Wilson 95% CI
UCB1-optimal vs baseline	66%	[56%, 75%]
UCB1-Tuned-optimal vs baseline	70%	[60%, 78%]
UCB1-optimal vs UCB1-Tuned-optimal	60% (n=100)	[50%, 69%]
UCB1-optimal vs UCB1-Tuned-optimal (n=400)	59.25%	[54%, 64%]

The paradox

UCB1-Tuned wins **more** against the baseline (70 vs 66), but **loses** the direct match against UCB1 (40 vs 60).

Explanation (winrate broken down by position):

Agent	As P1 (vs baseline)	As P2 (vs baseline)
UCB1-optimal (“spiky”)	94%	38%
UCB1-Tuned-optimal (“symmetric”)	82%	58%

UCB1-Tuned plays more defensively (variance-aware) → the weak baseline cannot exploit it, but the strong UCB1 can.

Dataset generation

Parallelised self-play

Methodology:

- MCTS-vs-MCTS, both with the optimal config
- Every MCTS move is labelled → **(state, move)** pair
- $\text{eps}=0.25$ (random exploration) + $\text{opening_max}=12$ (random openings)
→ diversify the dataset's states

Determinism:

- Per-game seed: $f(\text{base_seed}, \text{game_index})$
- → allows parallelising across machines with **bit-identical merge** (vs single-machine run)

Final dataset

Self-play games	150 000
Labelled positions	1 464 670
Features	46
Classes	14
Imbalance ratio	49.3×

Features: 42 board cells + 4 derived (move_count, current_player, P1 / P2 pieces on the bottom row).

We also generated a **baseline dataset** (with pre-tuning MCTS) to isolate the effect of MCTS tuning on the final tree's performance.

From-scratch implementation (no sklearn):

- **Entropy:** $H(y) = - \sum_c p_c \log_2 p_c$
- **Information Gain:**
 $IG(X) = H(y) - \sum_v \frac{|y_v|}{|y|} H(y_v)$
- Multi-way recursion (categorical) with pre-pruning
(max_depth, min_samples)

Iris warm-up (required by the spec):

- 4-bucket quantile binning per feature
- Test accuracy: **93.33%**
- Small tree (19 nodes, depth 4) → fully visualisable

Iris (warm-up)	
Rows	150
Features	4 (continuous)
Classes	3
Test accuracy	93.33%

PopOut	
Rows	1.46 M
Features	46
Classes	14
Test accuracy	<i>next slide</i>

Iris confirms our ID3 works before we apply it to the harder problem (PopOut).

ID3 — Methodological validation

Hyperparameter sweep across both datasets

Hyperparameters tuned:

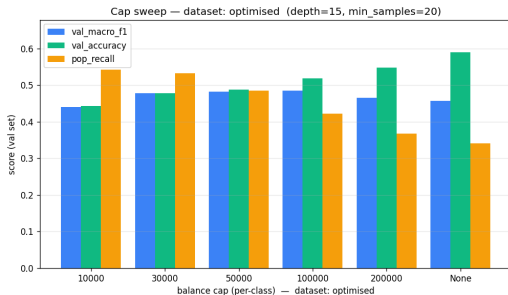
- `cap`: per-class limit (undersampling to reduce imbalance)
- `max_depth` and `min_samples`: pre-pruning of the tree

Sweep on both datasets (baseline + optimised):

- Sub-A: 6 values of `cap` (10k to full dataset)
- Sub-B: 5×4 grid (`depth` $\times$ `min_samples`)
- Cross-dataset transferability check: apply each dataset's best recipe to the other

Result:

- Recipe transfers well: gap ≤ 0.75 p.p. macro-F1
- Adopted shared recipe:
`cap=100k`, `depth=20`, `min_samples=10`



Cap sweep on the optimised dataset.

Macro-F1 plateaus at ~ 0.49 for cap between 30k–100k.

4 trees trained ($2 \text{ datasets} \times \{\text{unpruned, REP-pruned}\}$). Stratified 72/8/20 split, no sklearn.

5-fold cross-validation (mean $\pm$ std):

Tree	Macro-F1	Top-3 acc	POP recall	N nodes
Baseline unpruned	0.466 ± 0.001	0.665 ± 0.001	0.456 ± 0.001	131 077
Baseline pruned	0.475 ± 0.001	0.729 ± 0.001	0.445 ± 0.003	38 132
Optimised unpruned	0.463 ± 0.003	0.677 ± 0.002	0.412 ± 0.004	117 173
Optimised pruned	0.468 ± 0.002	0.736 ± 0.001	0.396 ± 0.004	35 921

REP post-pruning: reduces trees $\sim 4\times$ ($\sim 160\text{k} \rightarrow \sim 40\text{k}$ nodes) and raises val accuracy from ~ 0.49 to ~ 0.60 .

Test set in two views:

- **Balanced** (uniform dist. from cap): macro-F1 ≈ 0.48 for both pruned trees, accuracy 0.49–0.51
- **Unbalanced** (natural game dist.): macro-F1 collapses to 0.33–0.36, but **top-3 rises** to ~ 0.79 — the tree rarely errs outside the "top 3 best"

DTPlayer vs MCTS — results + 2nd non-transitivity

4 tournaments (DTPlayer with “nearest legal move” fallback — deterministic, never plays illegal):

Matchup	Winrate (A)	Wilson 95% CI	N
DT-optimised vs MCTS-optimised	44.5%	[37.8%, 51.4%]	200
DT-baseline vs MCTS-optimised	36.5%	[30.1%, 43.4%]	200
DT-optimised vs DT-baseline	0.0%	[0.0%, 1.9%]	200
DT-optimised vs Random	98.2%	[96.6%, 99.1%]	500

Non-transitivity in DT-policy space

The literature reports 5–20% for BC-vs-search; **DT-optimised reaches 44.5% vs MCTS** — a strong result. Yet vs DT-baseline it loses **0–200** (every game, both sides!).

Rock-paper-scissors pattern: vs MCTS, DT-optimised (44.5%) > DT-baseline (36.5%); head-to-head, DT-baseline (100%) > DT-optimised.

Why this is not a bug: both trees are deterministic given the board. 200 games → 2 distinct trajectories (one per P1/P2 assignment) repeated 100× each. The 0% is robust by construction.

Interpretation: the chain “stronger MCTS → better dataset → better DT” breaks at the last link. **Playing strength is not classification accuracy** — the 4 trees have CV macro-F1 within 1 p.p. but in actual play the baseline dominates.

Conclusions and future work

What we achieved:

- Robust MCTS with 2 variants and full empirical tuning
- Dataset of 1.46 M positions generated deterministically
- ID3 from-scratch with methodologically-validated pipeline on both datasets
- DTPlayer with robust fallback satisfying the “computer-vs-computer” scenario
- **Main findings:** (i) optimal c shifts with iter count; (ii) non-transitivity UCB1 vs UCB1-Tuned (MCTS); (iii) non-transitivity DT-optimised vs DT-baseline (downstream); (iv) classification accuracy $\neq$ playing strength

Limitations:

- $N=100-400$ games per matchup $\rightarrow$ relatively wide CIs
- Parameters `eps` and `opening_max` used at literature-standard values, without empirical tuning
- 5-fold CV only on the final trees (not on the hyperparameter sweep)

Future work:

- **Heuristic** rollouts in MCTS (use board knowledge instead of random moves)
- Remove **duplicate states** from the dataset before splitting (to avoid data leakage)
- Exploit **symmetry**: normalise the board to the current player's perspective

Thank you!

Questions?

Miguel Almeida (up202303926), Francisco Passos (up202303922), Pedro Resende (up202207418)
FCUP IA 2025/2026